

Variação de Hiperparâmetros do Decodificador em Segmentação de Imagens

Wesley Nogueira Galvão
Universidade Federal de São Carlos
(Dated: July 24, 2025)

Este relatório apresenta uma investigação sobre o impacto da variação dos hiperparâmetros do decodificador na arquitetura EncoderDecoder para segmentação de imagens, utilizando o dataset Oxford-IIIT Pet. Foram avaliadas diferentes configurações de strides e a inclusão de camadas extras no decodificador, analisando o desempenho de segmentação e o custo computacional de cada modelo.

I. INTRODUÇÃO

A segmentação de imagens representa uma tarefa fundamental em Visão Computacional, com aplicações em áreas como medicina, agricultura, veículos autônomos e robótica. O avanço das redes neurais convolucionais (CNNs) proporcionou melhorias significativas de desempenho, especialmente com arquiteturas EncoderDecoder, que combinam extração de atributos e reconstrução espacial para gerar máscaras de segmentação precisas.

A. Motivação

Decodificadores flexíveis possibilitam a adaptação da arquitetura para diferentes níveis de detalhe e custo computacional. A escolha dos strides e a inclusão de camadas extras impactam diretamente a capacidade do modelo de recuperar detalhes espaciais e a eficiência do treinamento. Métodos como o EncoderDecoder são especialmente relevantes em cenários nos quais a precisão da segmentação é crítica, como na detecção de tumores ou segmentação de objetos em ambientes complexos.

II. OBJETIVOS

1. Avaliar o impacto da variação dos hiperparâmetros do decodificador na qualidade da segmentação.
2. Comparar o custo computacional das diferentes configurações.
3. Discutir vantagens e limitações de cada abordagem.

III. METODOLOGIA

A. Conjunto de Dados

O Oxford-IIIT Pet Dataset, ilustrado na Figura III A, foi utilizado, e é composto por 7349 imagens de animais de estimação e suas respectivas máscaras de segmentação. O conjunto de teste contém 3669 imagens, enquanto 3680 imagens foram separadas para treino e validação (80% para treino e 20% para validação), com embaralhamento e semente fixa para garantir reprodutibilidade. Todas as

imagens possuem anotações trimap, com pixels classificados como 1: região do animal (foreground), 2: fundo (background) e 3: não classificados (pixels ignorados).

B. Arquitetura EncoderDecoder

A arquitetura EncoderDecoder consiste em um encoder ResNet50 pré-treinado no ImageNet, responsável pela extração de atributos em diferentes resoluções (strides 2, 4, 8, 16, 32), e um decodificador que reconstrói a máscara de segmentação a partir desses atributos. O decodificador pode ser configurado para utilizar diferentes combinações de strides e incluir camadas extras de convolução, batch normalization e ReLU.

C. Variações do Decodificador

As seguintes configurações foram avaliadas:

- Utilizar apenas os atributos no stride 32
- Utilizar apenas os atributos no stride 2
- Utilizar os atributos nos strides 2 e 32
- Utilizar os atributos nos strides 2, 8 e 32
- Utilizar os atributos nos strides 2 e 32 com camadas conv-batchnorm-relu extras em cada bloco do decodificador

D. Procedimento Experimental

Cada modelo foi treinado por 10 épocas, utilizando o encoder pré-treinado, batch size 128 para treino e 16 para validação, learning rate 1×10^{-2} e weight decay 1×10^{-3} . O desempenho foi avaliado pela métrica de IoU (Intersecção sobre a União) e pelo tempo total de inferência.

E. Teoria do Método

O EncoderDecoder combina extração de atributos em múltiplas escalas com reconstrução espacial. O encoder

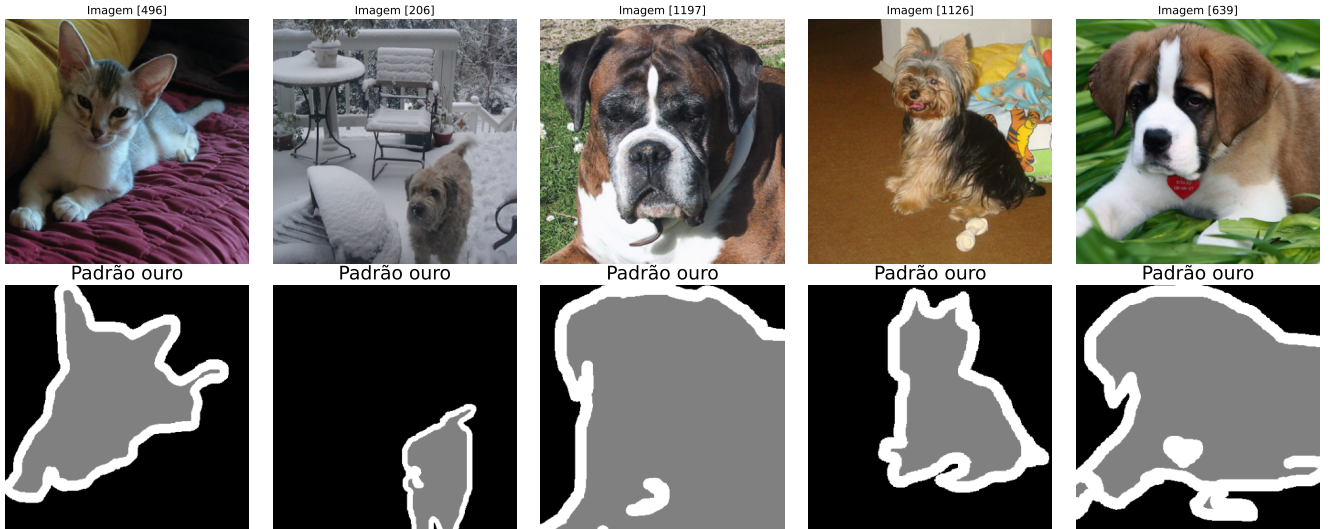


FIG. 1. Exemplo de imagens do conjunto de dados e suas respectivas máscaras de segmentação. As máscaras possuem três classes: fundo, região do animal e região de pixels a serem ignorados.

gera tensores de diferentes resoluções, que são combinados pelo decodificador para recuperar detalhes espaciais. A escolha dos strides define o nível de detalhe e o custo computacional. A inclusão de camadas extras no decodificador pode aumentar a capacidade de modelagem, mas também o custo computacional.

O processo de segmentação inicia-se pela extração das ativações intermediárias do encoder ResNet, realizadas pela função `get_features`. Essa função percorre as principais camadas do encoder, coletando tensores em diferentes resoluções (strides), que são utilizados pelo decodificador para reconstruir a máscara de segmentação.

Essas features correspondem aos mapas de ativação em múltiplas escalas, permitindo ao decodificador combinar detalhes locais e contexto global. O pseudocódigo abaixo ilustra esse processo.

```

1: Input: Imagem de entrada  $x$ 
2:  $x \leftarrow \text{conv1}(x)$ 
3:  $x \leftarrow \text{bn1}(x)$ 
4:  $x \leftarrow \text{relu}(x)$ 
5:  $\text{features.append}(x)$ 
6:  $x \leftarrow \text{maxpool}(x)$ 
7: for cada bloco do encoder (layer1 a layer4) do
8:    $x \leftarrow \text{bloco}(x)$ 
9:    $\text{features.append}(x)$ 
10: end for
11: Output: Lista de features em diferentes resoluções

```

No fluxo completo, as features extraídas por `get_features` alimentam o decodificador, que as processa conforme o pseudocódigo anterior, combinando-as para gerar a máscara segmentada final.

```

1: Input: Lista de tensores de atributos  $\text{features}$ , lista de strides  $S$ , número de canais  $C$ , flag  $\text{extra\_convs}$ 
2: Inverter ordem dos  $\text{features}$  e  $S$ 
3:  $x \leftarrow$  camada convolucional inicial em  $\text{features}[0]$ 

```

```

4: for  $i = 1$  até  $|\text{features}| - 1$  do
5:    $x \leftarrow$  interpolar  $x$  para resolução de  $\text{features}[i]$ 
6:    $x \leftarrow x +$  ajuste de canais de  $\text{features}[i]$ 
7:   if  $\text{extra\_convs}$  then
8:      $x \leftarrow \text{conv-batchnorm-relu extra}$ 
9:   end if
10: end for
11:  $\text{mask} \leftarrow$  camada de classificação em  $x$ 
12: Output:  $\text{mask}$ 

```

O parâmetro `extra_convs` permite adicionar uma camada extra de convolução, normalização e ativação (conv-batchnorm-relu) em cada bloco do decodificador. Essa camada adicional aumenta a capacidade de modelagem do decodificador, tornando-o capaz de capturar padrões mais complexos e refinar a reconstrução espacial das máscaras de segmentação.

IV. RESULTADOS

A. Métricas de Treinamento

Na Figura IV A, é possível observar o comportamento das curvas de train_loss , valid_loss e IoU para cada arquitetura de decodificador avaliada. Os modelos de alto desempenho — `strides_2_8_32`, `strides_2_32`, `strides_2_32_extra_convs` e `strides_32` — apresentam curvas de perda decrescentes e suaves ao longo das épocas, o que indica um processo de treinamento estável e eficaz. As curvas de validação acompanham de perto as de treino, sem sinais evidentes de overfitting, reforçando a capacidade de generalização desses modelos.

Entre eles, o modelo `strides_2_8_32` se destaca por atingir os menores valores de perda e os maiores valores de IoU, evidenciando uma aprendizagem mais eficiente,

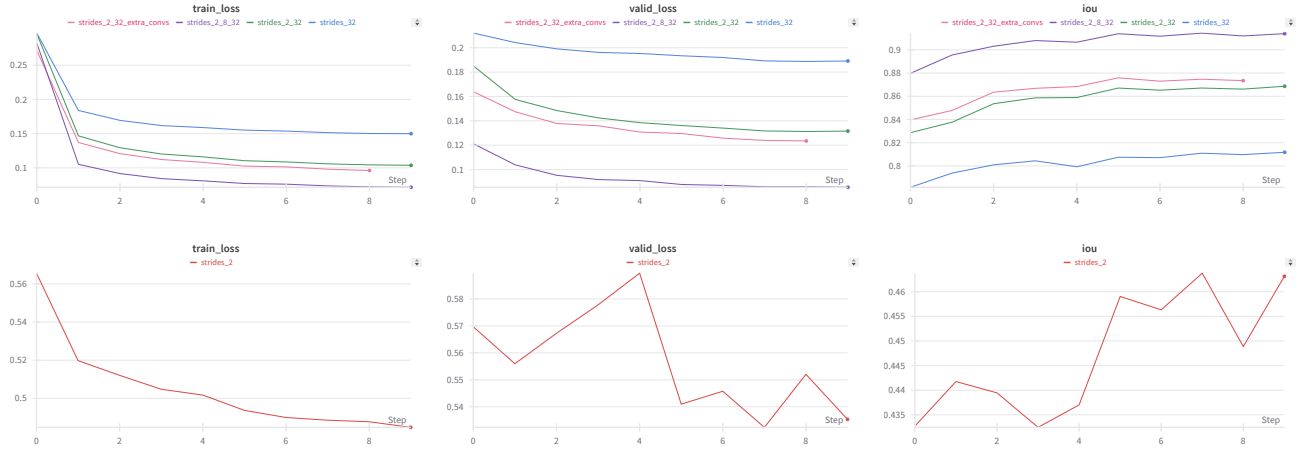


FIG. 2. Evolução das métricas de treinamento e validação ao longo das épocas.

dado que este modelo possui a combinação de mapas de ativação de diferentes resoluções.

Por outro lado, o modelo `strides_2` apresenta uma curva de `train_loss` elevada e instável, sugerindo dificuldade de aprendizado com apenas 10 épocas. No entanto, a curva de IoU apresenta uma tendência de subida, indicando que o modelo poderia melhorar com mais épocas de treinamento.

Em síntese, as métricas de treinamento demonstram que a combinação de múltiplos strides resulta em aprendizado mais eficaz e estável, enquanto o uso isolado de stride 2 se mostra inadequado para a tarefa de segmentação.

B. Segmentações e Comparação Qualitativa

A Figura IV B apresenta exemplos de segmentações geradas pelos diferentes modelos avaliados. Cada linha corresponde a uma configuração do decodificador, permitindo observar as diferenças qualitativas nas máscaras de segmentação produzidas. É notório que os modelos com múltiplos strides (como `strides_2_8_32` e `strides_2_32`) conseguem capturar melhor os detalhes dos objetos segmentados, resultando em máscaras mais precisas e com menos artefatos. Enquanto o modelo `strides_2` apresenta segmentações mais imprecisas, com falhas significativas na delimitação dos objetos, indicando confusão entre o fundo e o objeto segmentado. Já o modelo `strides_32`, apesar de ser capaz de segmentar os objetos, apresenta uma resolução mais baixa, resultando em máscaras menos detalhadas.

C. Análise do Desempenho Quantitativo

A Tabela I apresenta uma comparação direta dos modelos em métricas-chave de segmentação. O modelo `strides_2_8_32` destaca-se como o mais pre-

ciso, atingindo IoU médio de 0.729. Os modelos `strides_2_32_extra_convs` e `strides_2_32` apresentam desempenho semelhante, com valores próximos a 0.706 e 0.701, respectivamente. O modelo `strides_32`, embora funcional, apresenta resultado inferior (0.677), enquanto `strides_2` demonstra desempenho insatisfatório e alta variabilidade (IoU médio de 0.407, desvio padrão 0.200), evidenciando instabilidade.

Em relação à eficiência, todos os modelos de melhor desempenho possuem número de parâmetros semelhante, em torno de 26.7 a 26.8 milhões, e o tempo de inferência é praticamente constante entre eles. O modelo `strides_2_8_32` oferece o melhor equilíbrio entre precisão e custo computacional, já que o pequeno acréscimo no tempo de inferência é compensado pelo ganho em desempenho.

V. DISCUSSÃO

Os resultados indicam que a inclusão de múltiplos strides no decodificador melhora a qualidade da segmentação, especialmente ao combinar strides baixos (maior resolução) com altos (maior contexto). O uso de camadas extras proporciona ganhos marginais, mas aumenta o custo computacional. O modelo com strides 2, 8 e 32 apresentou o melhor IoU médio, evidenciando que a combinação de diferentes escalas é benéfica para capturar detalhes e contexto.

O tempo de inferência permaneceu similar entre os modelos, com pequenas variações atribuídas ao número de parâmetros. Todos os modelos utilizaram encoder pré-treinado, o que acelerou o treinamento e permitiu boa generalização mesmo com poucas épocas.

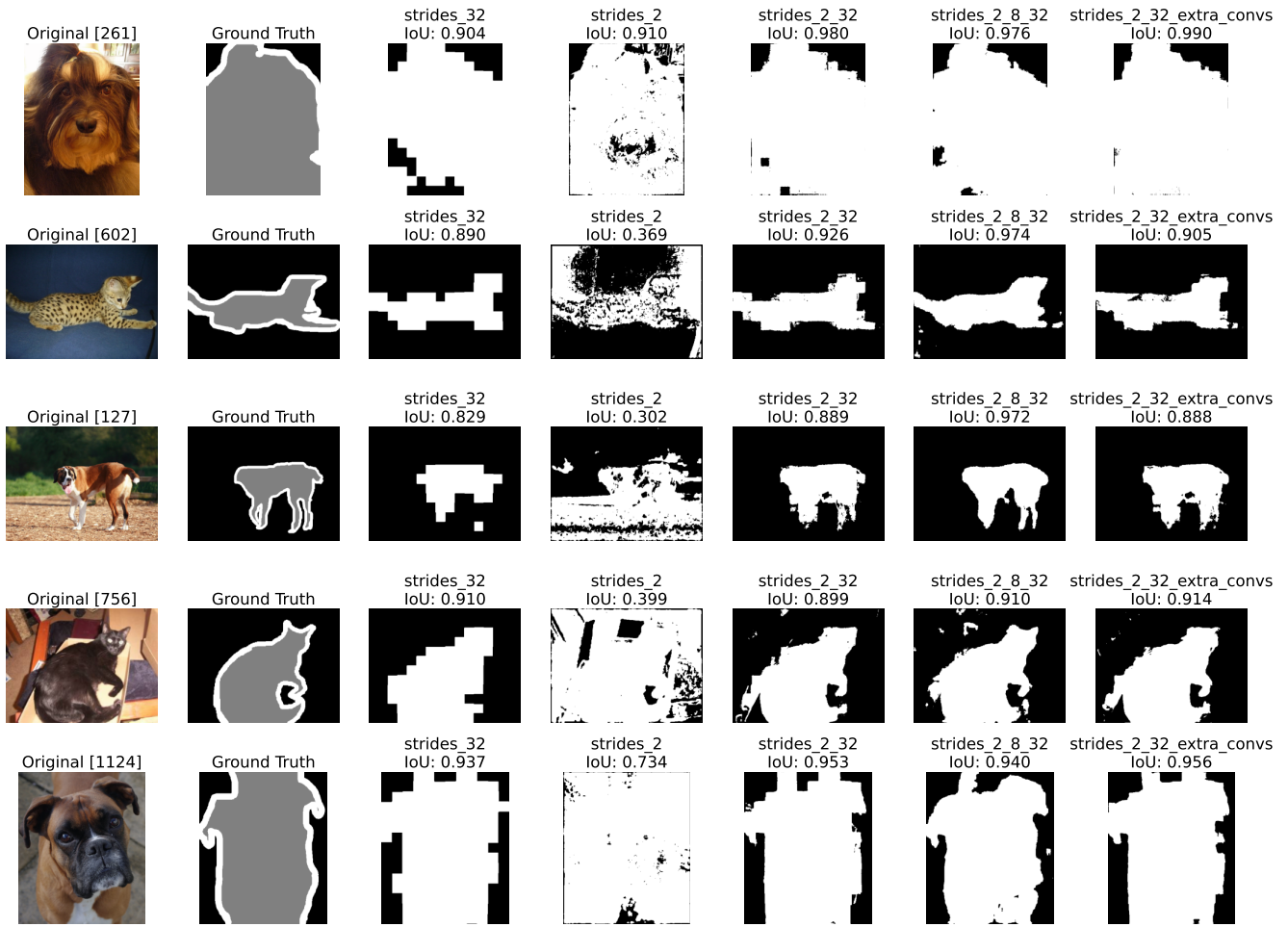


FIG. 3. Exemplo de segmentações geradas pelos diferentes modelos. Cada linha representa uma configuração do decodificador.

VI. CONCLUSÃO

O estudo realizado demonstra que a variação dos hiperparâmetros do decodificador impacta significativamente a qualidade da segmentação. Recomenda-se a combi-

nação de múltiplos strides para cenários nos quais detalhes e contexto são importantes. O uso de camadas extras pode ser explorado para ganhos adicionais, desde que o custo computacional seja aceitável. Como possibilidade futura, sugere-se investigar decodificadores mais profundos, atenção espacial e integração com arquiteturas modernas como U-Net e DeepLab.

TABLE I. Desempenho dos modelos de decodificador nas métricas de segmentação.

Modelo	IoU médio $\pm$ std	Parâmetros	Tempo total de inferência (s)
strides_32	0.677 ± 0.130	2.674×10^7	13.928
strides_2	0.407 ± 0.200	2.560×10^7	14.073
strides_2_32	0.701 ± 0.128	2.678×10^7	14.234
strides_2_8_32	0.729 ± 0.122	2.685×10^7	14.777
strides_2_32_extra_convs	0.706 ± 0.128	2.682×10^7	14.752